

Foreign Whispers Project Report Technical:

Technical Work

For this project, I mainly followed the instructions in the Phase 2 notebook and implemented each part step by step. Since I used my own MacBook Air with an M1 chip, I had to run everything on CPU instead of CUDA. Because of this, some parts were much slower than expected. I also modified a few package versions, such as torch and sentencepiece, so they could work better in my local environment. I tried to run part of the backend, especially ChatterboxTTS, on Colab, but the model would not load even after several changes. In the end, I decided to run the whole project locally on CPU.

Part 1: Video Download

This part was mostly straightforward. I did not run into any major issues when downloading the video.

Part 2: Transcription Integration

The transcription part also worked without major problems. I noticed that Whisper STT generated slightly longer transcripts with fewer segments compared to the YouTube captions. However, Whisper took a long time to run on my local CPU environment, which made debugging slower.

Part 3: Translation Integration

For the translation stage, I implemented a reranking function. Since I am not very familiar with Spanish, I used ChatGPT to help me understand some common Spanish language patterns. My approach was mostly rule-based. I added paraphrasing and phrase replacement rules to shorten some translated sentences while keeping the meaning close to the original.

For example, I replaced longer Spanish phrases with shorter alternatives, such as:

```
(r"\ben este momento\b", "ahora")  
(r"\bcon el fin de\b", "para")  
(r"\bdebido a que\b", "porque")  
(r"\bsin embargo\b", "pero")  
(r"\bpor supuesto\b", "claro")
```

After applying these shortening rules and some basic text cleaning, I loaded the Marian translation model: Helsinki-NLP/opus-mt-tc-big-en-es

The model generated translation candidates, and I returned the candidate list with the shortest sentence first. At this stage, I ranked shorter translations first because shorter Spanish sentences are easier to fit into the original timing windows. Later, I changed the logic so that candidates are ranked based on how close their estimated duration is to the target segment duration. I spent a lot

of time setting up and debugging the Marian model because there were several environment and dependency issues.

Part 4: Diarization Integration

For diarization, the first step was to align each transcription segment with a speaker. I did this by matching each segment to the speaker whose diarization interval overlapped the most with that segment. This part was fairly straightforward.

The next parts involved connecting diarization to the API, merging speaker labels into the transcription output, and making sure the frontend could display the speaker information correctly. I mostly followed the notebook instructions, but I also found that some files not clearly mentioned in the notebook still needed to be modified, such as files related to imports and shared variables.

For the voice assignment strategy, I used a round-robin method to assign different voices to different speakers. I also spent time checking that the frontend correctly reflected the diarization results. One issue I had to debug was related to caching, because old cached results sometimes made it look like the changes were not working.

Part 5: Alignment Integration

In the initial stretch-distribution visualization, the translated Spanish segments already fit the English timing windows reasonably well. I think this happened because the translation stage used both a stronger translation model and rule-based shortening. Before improving the duration estimator, only a small number of segments needed mild stretching, and no segments were classified as REQUEST_SHORTER or FAIL.

For Task 1, I improved the TTS duration estimator. The original estimator used a simple syllable-rate heuristic. I replaced it with a lightweight regression model trained on actual Chatterbox TTS durations from the alignment reports. The model predicts duration using features such as character count, syllable count, punctuation count, and word count.

The regression-based estimator worked much better than the original heuristic. The previous syllable-based estimate had a mean absolute error of about 1.22 seconds, while the regression-based predictor reduced the error to about 0.18 seconds on the available TTS samples. After applying this improved estimator, all 170 segments were classified as ACCEPT by the alignment policy.

For Task 2, I implemented duration-aware translation reranking as a fallback for segments that are too long. In the current clip, there were no REQUEST_SHORTER or FAIL segments, so the reranker was not triggered in the main evaluation. However, I still added candidate ranking based on timing fit. The reranker uses the predicted duration from Task 1 and prefers candidates whose estimated TTS duration is closest to the target duration. I also added filtering to avoid incomplete or clipped candidates.

For Task 3, I implemented a simple beam-search global alignment method as an alternative to the greedy optimizer. The greedy optimizer makes local decisions segment by segment, while beam search keeps multiple possible alignment paths and selects the lowest-cost final schedule. In this clip, the greedy optimizer already performed well because all segments were accepted and no major gap shifts were needed. Because of that, both greedy alignment and beam search produced no additional drift or severe stretch cases. For the final video alignment, I kept the original greedy method because it was already sufficient, although beam search could be more useful for harder examples.

For Task 4, I built a Dubbing Quality Scorecard to evaluate dubbing quality across several dimensions, including timing accuracy, naturalness, intelligibility, semantic fidelity, and overall score. Since I did not run a real Spanish TTS-to-STT round trip, intelligibility is reported as None instead of being treated as a measured value. For semantic fidelity, I used a multilingual Sentence Transformers model to embed the English source text and Spanish translation, then computed cosine similarity.

The final scorecard was:

```
timing_accuracy: 0.6
intelligibility: None
semantic_fidelity: 0.865
naturalness: 0.792
overall: 0.752
```

The semantic fidelity score suggests that the Spanish translations preserved the source meaning reasonably well. The timing score was lower than I expected because the current scorecard still penalizes some underfilled or overlap-related cases, even though the alignment policy classified all segments as ACCEPT. This shows that the scorecard could be improved in the future, especially by focusing more on overflow problems rather than absolute duration differences.

Part 6: TTS Integration

This part focused on per-speaker voice assignment. It took me a lot of time to debug because I originally tried to run diarization and TTS through Colab, but the setup kept failing. After that, I decided to run the full pipeline locally.

Task 1 was mainly about understanding how the existing TTS pipeline worked. Task 2 required implementing a voice resolution function with the priority order:

speaker-specific WAV > default language voice > global voice

Task 3 added a speaker_wav parameter to the TTS API. Task 4 implemented the actual per-speaker voice assignment. The coding part itself was not too difficult, but it required carefully passing the correct variables through the pipeline. One important line was:

```
"speaker_wav": (speaker_voice_map or {}).get(seg.get("speaker")) or speaker_wav
```

This allowed each segment to use the correct speaker-specific voice when available, while still falling back to the default voice if needed.

Part 7: Stitch Integration

This part did not require much new coding. The main goal was to check whether the final dubbed audio and video could be stitched together correctly.

At first, my dubbed audio sounded very bad, and I thought the TTS system was not working properly. Later, I realized that I had not installed the rubberband package. After installing it, the dubbed video finally worked much better. This was a good reminder that audio-processing dependencies are very important, and missing one package can make the output seem completely broken.

Overall Reflection

Overall, I found this project really interesting. I spent a lot of time debugging, deleting caches, reinstalling packages, checking whether the API was alive, and trying to understand why some parts worked locally but not in Colab. Even though this was frustrating at times, I learned a lot from the process.

One part I enjoyed was seeing how small pieces of code gradually came together into a full application that can translate and dub a video into another language. I also learned more about frontend integration, audio processing, speaker diarization, translation models, TTS, and alignment.

The biggest limitation was that I could not test the project in a CUDA environment. Because of that, I could not try some larger or more advanced translation and audio models. I also started the project later than I should have, so I did not have enough time to experiment with different models or use an LLM API more deeply.

Even with these limitations, I still learned a lot from this project. It helped me understand how complicated video dubbing systems are, especially when translation, timing, speaker identity, and audio quality all need to work together. I think this topic is worth exploring more in the future.